WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

仓颉兴趣组项目分享

主讲人：孟世博

HTTP 客户端

Web 框架模板引擎

MIME 类型

工具库



目录

- 关于仓颉兴趣组
- ginger
- tea template
- filetype4cj
- xurls4cj
- dateparse
- tabulate4cj
- handy4cj

SIGCANGJIE / 仓颉兴趣组 是来自**东北大学软件学院**的开源社区。

- **学生驱动，学生所有**
- 所有的 Credit 都由代码的**原始贡献者**所有
- 已有 **21** 个 **Cangjie-TPC / Cangjie-SIG** 项目

naivetoolkit
AN OPEN SOURCE PROJECT OF SIGCANGJIE

- **naivetoolkit** - 轻量级云原生开发套件

MEGA PROJECT
OF SIGCANGJIE

- **cjgrapht** - 图数据结构 & 算法库
- **cjnum** - 数值分析库

XPERIMENTAL PROJECT
OF SIGCANGJIE

- **spu4cj** - 用于流处理应用的嵌入式DSL

Ginger 是一个轻量级 HTTP 客户端。支持：

- 自定义请求配置
  - 全局配置
  - 局部配置
- 自定义**拦截器**
- **同时**支持仓颉 0.53.18 和 0.59.6 版本

快速发起请求

```
import ginger.default.*

main(): Int64 {
  let response = get(
    "http://jsonplaceholder.typicode.com/todos/1"
  )
  println(response.readString())
  return 0
}
```

使用客户端发起请求

```
import ginger.*

main(): Int64 {
  let ginger = Ginger()
  let response = ginger.get(
    "http://jsonplaceholder.typicode.com/todos/1"
  )
  println(response.readString())
  return 0
}
```

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

发起带 Body 的请求 & 响应序列化

```
import ginger.default.*
import serialization.serialization.*

// generated by json2cangjie.cn
class Post <: Serializable<Post> & ToString {
  public Post(
      public var id: Int32,
      public var title: String,
      public var body: String,
      public var userId: Int32
  ) {}
// ...
}
```

```
main(): Int64 {
  let response = post(
    "http://jsonplaceholder.typicode.com/posts",
    body: RequestBody.of(Post(
      "cangjie",
      "cangjie workshop",
      2
    ))
  )

  println(response.readAs<Post>())
  return 0
}
```

```
{"id": 101, "title": "cangjie", "body": "cangjie workshop", "userId": 2}
```

自定义请求配置

```
package ginger_demo

import ginger.*

main(): Int64 {
  let ginger = Ginger(config: RequestConfig(
    baseUrl: "http://jsonplaceholder.typicode.com/"    ❶
  ))
  let response = ginger.get(
    "/todos/1",
    config: RequestConfig(
      headers: [("Custom-Header", ["value"])]          ❷
    )
  )
  println(response.readString())
  return 0
}
```

❶ 全局配置 – 实例级别生效

❷ 局部配置 – 单次请求生效

自定义拦截器 – 以实现 Mock 客户端为例

```
main(): Int64 {
  let mockClient = Ginger()

  mockClient.addInterceptors { ctx =>
    let dm = match (ctx.body.getOrThrow()) {
      case Model(dm) => dm
      case _ => return
    }
    let newPost = Post.deserialize(dm)
    newPost.id = 102
    ctx.response = HttpResponseBuilder()
      .status(200)
      .header("Content-Type", "application/json")
      .body(newPost.toString())
      .build()
  }
```

```
  let response = mockClient.post(
    "http://jsonplaceholder.typicode.com/posts",
    body: RequestBody.of(Post(
      "cangjie", "cangjie workshop", 2
    ))
  )
  println(response.readAs<Post>())
  return 0
}
```

拦截请求并返回自定义响应

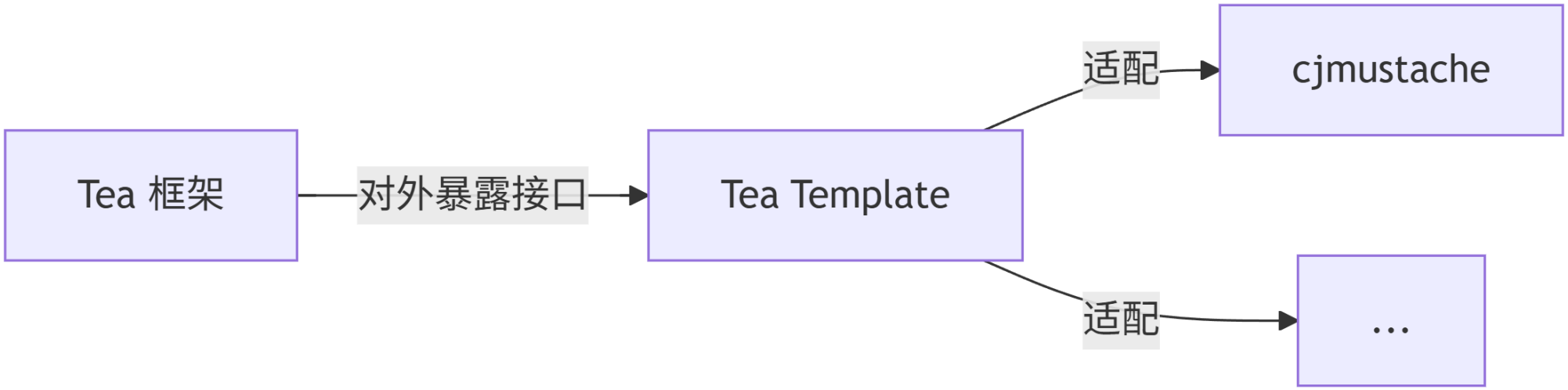
```
{"id": 102, "title": "cangjie", "body": "cangjie workshop", "userId": 2}
```

Tea Template

Tea Template 为 Web 框架 **Tea** 提供了**模板引擎**支持。

- 目前支持的模板引擎：
  - **cjmustuche**, https://gitcode.com/Cangjie-SIG/cjmustache
- 基于 DataModel 渲染，**不依赖反射**
- 支持仓颉 0.59.6 版本

Tea Template 与 Tea 框架的关系

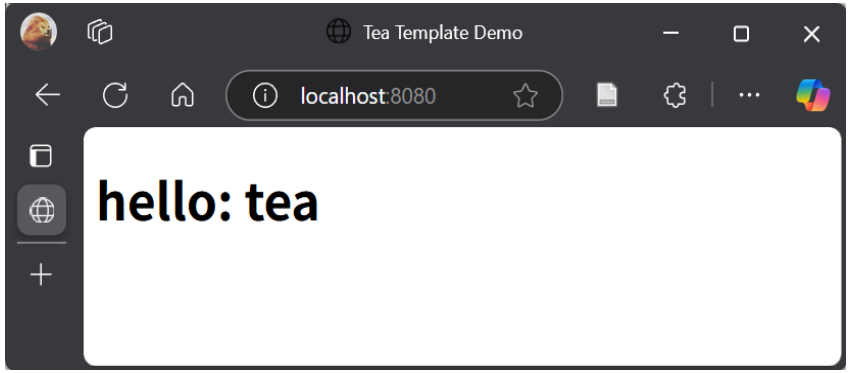


快速开始 – 以 cjmastache 为例

```
import tea.*
import tea_template.mustache.MustacheEngine
import serialization.serialization.*
import std.collection.*

main(): Int64 {
  let templateEngine = MustacheEngine(
    directory: "./assets",
    extension: "html"
  )
  let app = Tea.new(TeaConfig(views: templateEngine))
  app.get("/") { ctx =>
    ctx.render("index", HashMap(("name", "tea")).serialize())
  }
  app.run(8080)
  return 0
}
```

```
<!-- index.html -->
<!DOCTYPE html><head>
    <title>Tea Template Demo</title>
</head>
<body>
    <h1>hello: {{ name }}</h1>
</body>
</html>
```



复杂示例 - https://gitcode.com/naxida/tea_template_demo




```
{
  "title": "复杂模板示例",
  "company": {"name": "TeaTemplate"},
  "navItems": [
    {"text": "首页", "link": "#"},
    {"text": "功能", "link": "#features"},
    {"text": "评价", "link": "#testimonials"},
    {"text": "联系我们", "link":
"https://gitcode.com/SIGCANGJIE"}
  ],
  "hero": {
    "title": "现代化的模板渲染解决方案",
    "description": "TeaTemplate 提供易于使用的模
板渲染系统",
  …
  },
  …
}
```

filetype4cj 支持通过**魔数**推断文件的**类型**。

- 推断文件的**媒体类型**和**扩展名**
- 支持**数组**和**文件**的判断
- 自定义匹配器
- 支持仓颉 0.53.18 版本



获取文件的 MIME 类型 & 扩展名

```
import filetype4cj.*

main(): Int64 {
  // 扩展名为 .txt 的 JEPG 图片
  let res = matchFile("a.jpg.txt")
  println("master type: ${res.mime.masterType}")
  println("sub type:    ${res.mime.subtype}")
  println("mime type:   ${res.mime.value}")
  println("extension:   ${res.extension}")
  return 0
}
```

```
master type: image
sub type:    jpeg
mime type:   image/jpeg
extension:   jpg
```

```
import filetype4cj.*

main(): Int64 {
  // zip 压缩包魔数
  let res = doMatch([0x50, 0x4b, 0x03, 0x04])
  println("master type: ${res.mime.masterType}")
  println("sub type:    ${res.mime.subtype}")
  println("mime type:   ${res.mime.value}")
  println("extension:   ${res.extension}")
  return 0
}
```

```
master type: application
sub type:    zip
mime type:   application/zip
extension:   zip
```

自定义匹配器

```
import filetype4cj.*

main(): Int64 {
  let java = addType(
    "class",
    "application/java-vm"
  )

  addMatcher(java) { arr =>
    return arr.slice(0, 4) == [0xca, 0xfe, 0xba, 0xbe]
  }
  let res = matchFile("A.class")
  println("master type: ${res.mime.masterType}")
  println("sub type:    ${res.mime.subtype}")
  println("mime type:   ${res.mime.value}")
  println("extension:   ${res.extension}")
  return 0
}
```

❶

❷

❶ 注册自定义类型

❷ 为注册的类型添加匹配器

```
master type: application
sub type:    java-vm
mime type:   application/java-vm
extension:   class
```

xurls4cj 使用**正则表达式**从文本中提取 **URL**。

- 多种匹配模式
  - 严格模式 – 精确匹配 URL
  - 宽松模式 – 匹配 IP 地址、邮箱等
  - 匹配特定 scheme
- 基于 RFC 3987 和 RFC 4291 标准，支持ASCII与Unicode域名
- 兼容多种官方与非官方URL scheme
- 支持仓颉 0.53.18 版本

严格匹配、宽松匹配和按照 scheme 匹配

```
import xurls4cj.*

main(): Int64 {
  let text = """
    For inquiries, email us at contact@my-company.io.   Check out our site at www.test-site.com.
    Server logs show connections from 172.16.254.1 and 2001:0db8:85a3::8a2e:370:7334.
    You can visit https://www.example.com or http://test-site.com/page?query=1.
  """
  println(strictAll(text))
  println(relaxedAll(text))
  println(strictMatchingSchemeAll("http", text))
  return 0
}
```

```
[https://www.example.com, http://test-site.com/page?query=1]
[https://www.example.com, http://test-site.com/page?query=1, www.test-site.com, www.example.com, test-
site.com/page?query=1, contact@my-company.io, 172.16.254.1, 2001:0db8:85a3::8a2e:370:7334]
[http://test-site.com/page?query=1]
```

dateparse

dateparse 能够**自动**识别并解析多种格式的日期字符串。

- 无需指定模板，支持**几十种**日期时间格式
- 支持处理时区
- 提供严格模式避免歧义
- 支持仓颉 0.53.18 版本

自动解析日期时间字符串

```
import dateparse.*
import std.collection.*

main(): Int64 {
  let dates = [
    "2006-01-02T15:04:05+0800",          // ISO 8601
    "Mon Jan  2 15:04:05 2006",          // Go
    "Mon, 02 Jan 2006 15:04:05 -0700",   // 带时区
    "2012/03/19 10:11:59.3186369",       // yyyy/mm/dd
    "20140722105203",                    // 紧凑格式
    "13842163671112223333",              // 时间戳
    "2025-04-29"                         // 仅日期
  ]
  dates |> forEach { s =>
    println(DateParse.default.parseAny(s, None, None))
  }
  return 0
}
```

```
2006-01-02T15:04:05+08:00
2006-01-02T15:04:05+08:00
2006-01-02T15:04:05-07:00
2012-03-19T10:11:59.3186369+08:00
2014-07-22T10:52:03+08:00
2013-11-12T00:32:47.111222333+08:00
2025-04-29T00:00:00+08:00
```

严格模式

```
import dateparse.*

main(): Int64 {
  try {
    let date = DateParse.default.parseStrict(
      "04/05/2023",
      None, None
    )
    println("解析成功: ${date}")
  } catch (e: ExceptionAmbiguous) {
    println("出现歧义: ${e}")
  }
  return 0
}
```

```
出现歧义: Exception: This date has ambiguous mm/dd
vs dd/mm type format
```

配置歧义时解析方式

```
import dateparse.*

main(): Int64 {
  let date = DateParse.default.parseAny(
    "04/05/2023",
    None, [
      // 优先将第一个数字解析为日期，而非月份
      ParserOptions.preferMonthFirst(false)
    ]
  )

  println(date)
  return 0
}
```

```
2023-05-04T00:00:00+08:00
```

tabulate4cj 能够轻松打印出美观的文本表格。

- 多种表格样式
- 支持自动换行、单元格最大宽度等配置
- 支持汉字、Emoji 等 Unicode 字符
- 支持仓颉 0.53.18 版本



打印表格 & 多种渲染配置

```
import std.collection.*
import tabulate4cj.tabulate.*

// 实现了 ToString 或 Formatter 即可表格化输出
class Demo <: ToString { … }

main() {
  let data: Array<Array<ToString>> = [
    ["ID", "demo", "note", "empty"],
    [1, Demo("test", 56.8791), "short note"],
    [2, Demo("apple", 56.8791), "This is a really
long string, yaaaay it works, Vivamus laoreet
vestibulum pretium."],
    [3, Demo("test", 56.8791), "111"],
    ["合计💰", 168]
  ]
  let tab = create(data)
  // 设置单元格最大宽度
  tab.setMaxCellSize(20)
  // 启用自动换行
  tab.setWrapStrings(true)
  // 换行时按空格分割
  tab.setWrapDelimiter(r' ')
  // 填补换行产生的空位
  tab.setSplitConcat("==")
  // 设置替换空单元格的字符串
  tab.setEmptyString("empty")
  // 设置数据内容为右对齐
  tab.setAlign("right")
  // 设置标题以及他的对齐方式
  tab.setTitle("Demo Case Info", align: "center")
  // 以grid样式渲染表格
  println(tab.render(format: "grid"))
}
```

打印结果

```
+-----------+--------------------+-------------------------+-----------+
|                          Demo Case Info                              |
+-----------+--------------------+-------------------------+-----------+
|        ID |                demo |                    note |     empty |
+===========+====================+=========================+===========+
|       001 |     test:56.879100 |              short note |     empty |
+-----------+--------------------+-------------------------+-----------+
|       002 |    apple:56.879100 |       This is a really  |     empty |
|           |                    |    long string, yaaaay  |     empty |
|           |                    |      it works, Vivamus  |     empty |
|           |                    |     laoreet vestibulum  |     empty |
|           |                    |                pretium. |     empty |
+-----------+--------------------+-------------------------+-----------+
|       003 |     test:56.879100 |                     111 |     empty |
+-----------+--------------------+-------------------------+-----------+
|    合计💰 |                168 |                   empty |     empty |
+-----------+--------------------+-------------------------+-----------+
```

为其他库提供美观的表格格式化

```
import dataframe_cj.*
import dataframe_cj.imports.*
import std.io.*

main(): Int64 {
  let json = ByteArrayStream.fromString("""
    ["id", "name", "account", "active"]
    [1, "Amy", 12.5, true]
    [2, "Bob", null, false]
    [3, "Cat", -3.8, null]
    [4, "Dan", 0.0, true]
    [5, "Eve", 150.0, false]
    [6, "Fay", null, null]
  """)
  let df = loadFromJson(json).getOrThrow()
  println(df.table())
  return 0
}
```

https://gitcode.com/zhangyin_gitcode/dataframe_cj

```
+------+------+---------+---------------+-----------+
|      |   id |    name |       account |    active |
+======+======+=========+===============+===========+
|    0 |    1 |     Amy |     12.500000 |      true |
+------+------+---------+---------------+-----------+
|    1 |    2 |     Bob |          None |     false |
+------+------+---------+---------------+-----------+
|    2 |    3 |     Cat |     -3.800000 |      None |
+------+------+---------+---------------+-----------+
|    3 |    4 |     Dan |      0.000000 |      true |
+------+------+---------+---------------+-----------+
|    4 |    5 |     Eve |    150.000000 |     false |
+------+------+---------+---------------+-----------+
|    5 |    6 |     Fay |          None |      None |
+------+------+---------+---------------+-----------+
```

handy4cj 为仓颉提供了多种实用工具。

- 环境变量使用增强
- 随机数组、字符串生成
- 数组、时间处理工具
- 字符串处理工具
- 支持仓颉 0.53.18 版本



环境变量增强

```
import handy4cj.*

main(): Int64 {
    // $env:CANGJIE_HOME = /cangjie-0.53.18
    // $env:NUMBER = 429
    // $env:PATH = foo;bar;...

    // 输出 /cangjie-0.53.18
    println(envStr("CANGJIE_HOME", "no cj"))
    // 输出 429
    println(envInt64("NUMBER", -1))
    // 输出 [foo, bar, ...]
    println(envStrs("PATH", ";", ArrayList()))
    // 输出 nothing
    println(envStr("NOTHING", "nothing"))

    return 0
}
```

随机数组 & 随机字符串生成

```
// 随机生成 5 个随机数，范围在 0 到 256 之间
// 输出 [246, 116, 31, 154, 202]
println(randomIntArray(0, 256, 5))

// 随机生成 4 到 8 位数字字母组成的字符串
// 输出 nQx48l
println(randomString(
  4, 8, false, true, false, false
))
```

字符串占位符格式化

```
// 输出 f90b91c7-1caf-82b5-0e21-d2849fe9706c
println(reshape(
  "########-####-####-####-############",
  "f90b91c71caf82b50e21d2849fe9706c"
))
```

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

欢迎访问 **仓颉兴趣组** 首页

http://www.sigcangjie.cn

